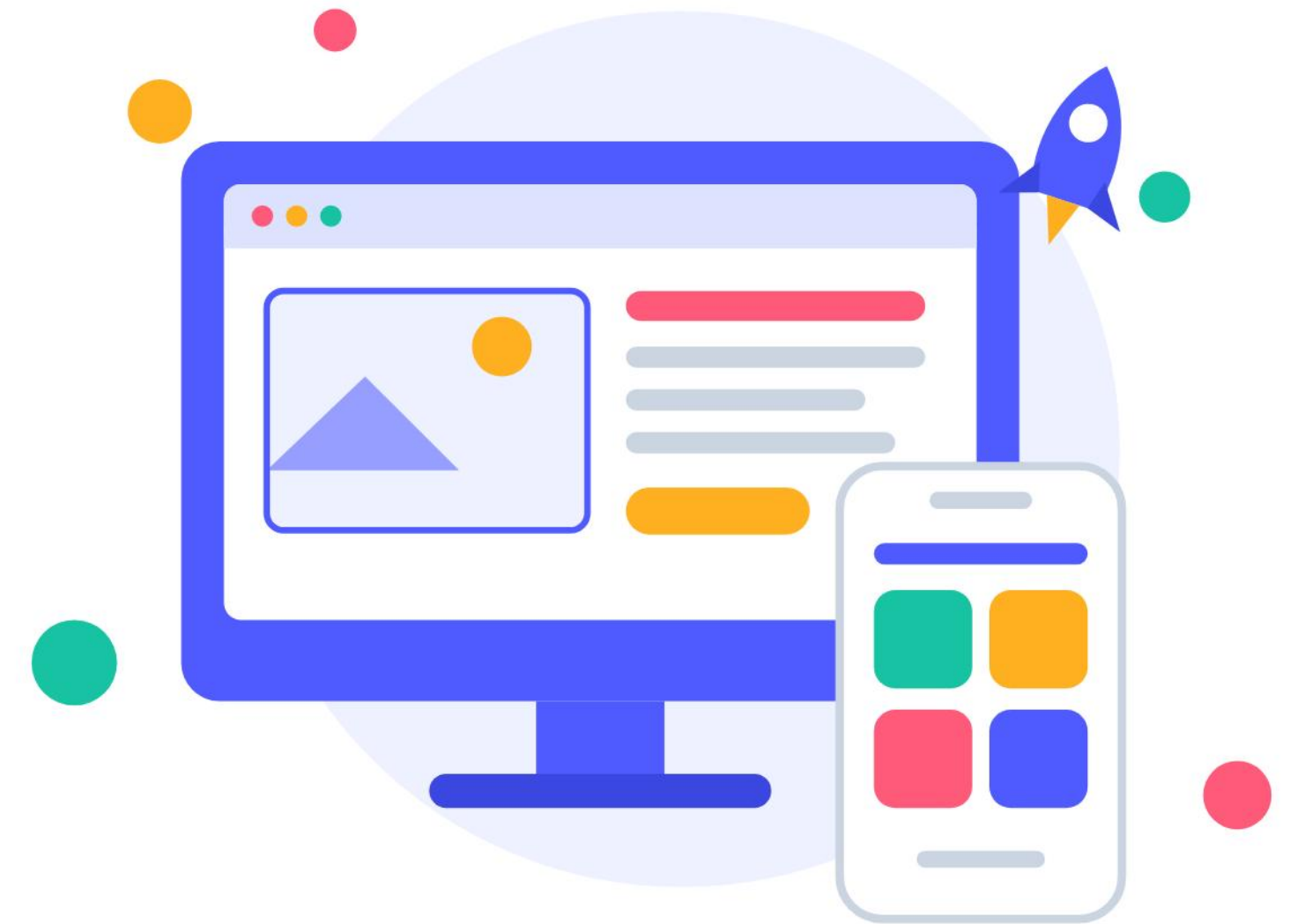


WEEK 05 · GRID · RESPONSIVE

THE PAGE GETS A SECOND DIMENSION

CSS Grid, and a page that fits every screen on purpose.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN



WHERE WE ARE

Week 5 of 12 (week 13 is the exam). You can put two things side by side. You cannot yet make a page.

THE NEXT 180 MINUTES

Five cycles, you typing in every one (the few long blocks are marked **copy this**): the shell Flexbox could not hold · a picture made of tracks · a card row with no query · one prompt to Claude · the shell again, from the phone up. Break at 90. The assignment at 145.

WHAT THIS UNLOCKS

Every page you build from here. And next week Tailwind just renames it.

TARGET · MINUTE 0

THIS IS WHERE YOU WILL BE AT MINUTE 180

the class assignment – resize the frame and watch it redraw

Signal

SERVICES

Gateway

Checkout

Search

Mailer

Reports

Gateway

Everything on this page is one grid holding five regions, and one grid inside it holding a row of cards that nobody counted in advance.

REQUESTS

1.42M

Across all regions in the last twenty-four hours, including the retries that the gateway absorbed on the client's behalf.

updated 2 min ago

LATENCY

86ms

ERRORS

0.31%

Activity

09:14 Deploy 4c81f2 finished

08:02 Worker pool scaled to 24

07:47 Alert cleared: eu-central latency

22:10 Certificate renewed

Signal

SERVICES

Gateway

Checkout

Search

Mailer

Reports

Gateway

Everything on this page is one grid holding five regions, and one grid inside it holding a row of cards that nobody counted in advance.

REQUESTS

1.42M

Across all regions in the last twenty-four hours, including the retries that the gateway absorbed on the client's behalf.

updated 2 min ago

LATENCY

86ms

Median, end to end.

updated 2 min ago

ERRORS

0.31%

Fives only. Fours are counted separately because most of them are somebody else's bug and not ours.

updated 2 min ago

SATURATION

61%

Peak worker pool.

updated 5 min ago

UPTIME

99.98%

Rolling thirty days.

updated 1 h ago

Activity

09:14 Deploy 4c81f2 finished

08:02 Worker pool scaled to 24

07:47 Alert cleared: eu-central latency

22:10 Certificate renewed

targets/week-05.png · 1280

Signal

Overview

Activity

Warm-ups

Gateway

Everything on this page is one grid holding five regions, and one grid inside it holding a row of cards that nobody counted in advance.

375 · the same five regions, one column

► **THE POINT** — five regions, three arrangements. The card row decides its own column count. There is not one `max-width` in the file. You type nearly every piece of it today — the few long blocks are marked **COPY THIS** — once in a cycle, once in the assignment.

AI THIS WEEK

TUTOR. IT EXPLAINS; YOU TYPE.

LAYER · WEEKS 1–5 · PRIVATE TUTOR

ALLOWED

Ask it why a column overflows, why a pattern comes out in stripes. Ask it to criticise CSS *you* typed. Then check the answer in DevTools, at more than one width.

NOT ALLOWED

Pasting generated CSS into the assignment. **You type nearly every piece.** Long blocks are marked *copy this*; the rest is yours. The skill this week: ask for a *rule of thumb* about your own grid, write it down as one sentence that could be false, then test it at the edges of your page — other widths, other conditions, a long word.

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Cycle 4 is one prompt, twice, in whatever chat you have.

THIS IS WHAT YOU BUILT LAST WEEK

the page shell, attempted in honest Flexbox

reload

open

☐ add a gap between the regions

☐ shrink the percentages to compensate

☐ give the main region more content

FIVE REGIONS, ONE PAGE SHELL, FLEXBOX ONLY

masthead

sidebar
25%

main — 50%

aside
25%

footer

The sidebar stops 48px above the footer.

container inner width 1498px

the band wants 1498px

gutter, sidebar to main 0px

sidebar bottom to footer 48px

wrapper elements needed 2

Nothing is wrong yet, and you should say so. Three percentages that add up to 100 do lay out three columns, and week 4 got you here honestly.

Now ask for the two things every real designer asks for: space between

► **HANDS-ON** `demos/01-why-grid/01-flex-attempt/` — tick **ADD A GAP**, then **MORE CONTENT**. Do not defend it.

4

CYCLE 1 · BEFORE YOU TYPE

GRID: YOU DEFINE THE CONTAINER FIRST

grid

A layout with rows *and* columns at the same time. Flexbox has one axis; Grid has two.

grid container The element you write `display: grid` on. Each of its direct children is a **grid item**.

track

One column or one row.
`grid-template-columns` lists the columns, `grid-template-rows` the rows. A **cell** is where a row and a column cross.

gap

Space between tracks, as in Flexbox.

```
1 .page {  
2   display: grid;  
3   grid-template-columns: 200px 200px 200px;  
4   gap: 16px;  
5 }
```

Three sizes, three column tracks; the items fill the cells in order. Alone, `display: grid` gives **one** column, which looks as if nothing happened.

TRACKS AND LINES

DEEPER · SELF-STUDY

EIGHT LINES OF CSS, AND NO WRAPPER ELEMENTS

the same five regions, in Grid

↺ reload

open ↗

Show me the CSS that does this

THE SAME FIVE REGIONS, AND NO WRAPPER ELEMENTS

masthead

sidebar

main

The main region holds more than the others, exactly as before. Nothing about that is a problem any more.

aside

footer

Both gutters are 12px. The sidebar meets the footer exactly.

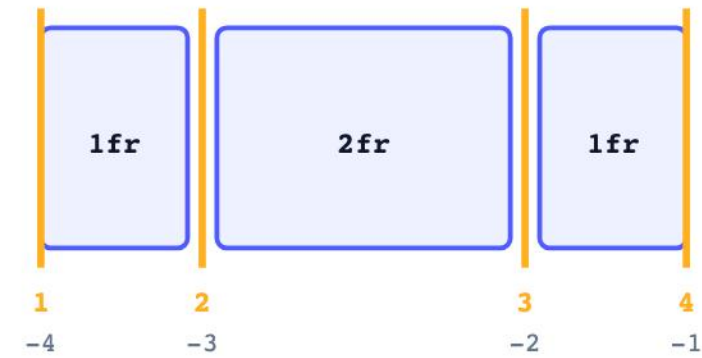
```
container inner width  1498px
the grid wants        1498px
gutter, sidebar to main 12px
sidebar bottom to footer 12px
wrapper elements needed 0
the browser resolved 1fr 2fr 1fr to
                        368.5px 737px 368.5px
```

► **HANDS-ON** `demos/01-why-grid/02-grid/` — guess the line count **BEFORE** you press the button.

CYCLE 1 · BEFORE YOU TYPE

ITEMS SIT BETWEEN LINES, NOT IN CELLS

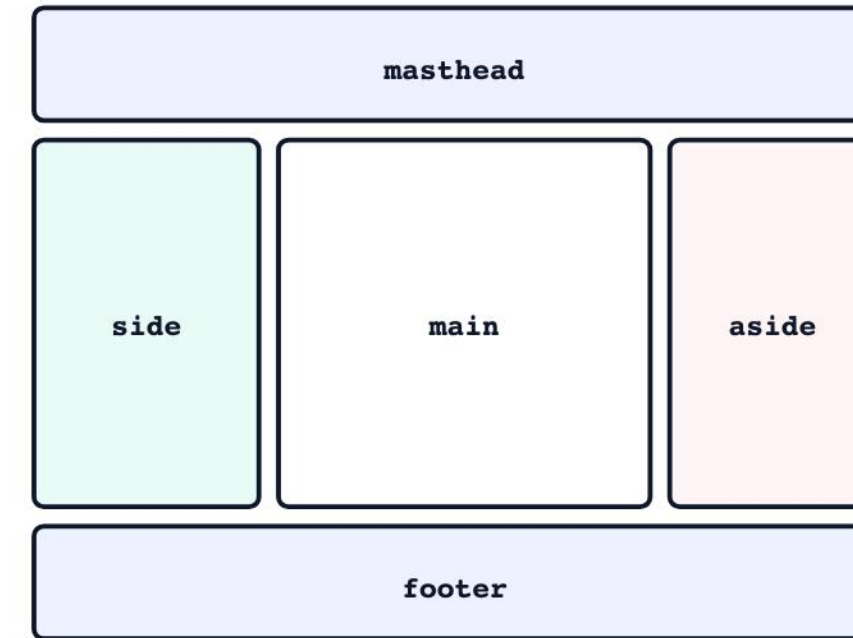
tracks, and the lines that come with them



an item sits between two lines



or draw the layout, and let the items claim a name



```
"masthead masthead masthead"  
"side    main    aside   "  
"footer  footer  footer  "
```

Three tracks. Four lines. The same line counted backwards — and `span 2`, which counts *tracks*, not lines. The drawing on the right comes back in cycle 5.

- A **LINE** is the edge of a track, numbered from 1. **THREE TRACKS, FOUR LINES** — count the edges, not the cells. `grid-column: 1 / -1` runs from the first line to the last, however many tracks there are; `grid-column: span 2` means two tracks wide.

CYCLE 1 · BEFORE YOU TYPE

fr: A SHARE OF WHAT IS LEFT

PERCENT IGNORES THE GAP

```
1 .row {  
2   grid-template-columns: 25% 50% 25%;  
3   gap: 16px;  
4 }
```

100% of the container, plus two gaps of 16px: **32px too wide.**

FR SHARES THE REST

```
1 .row {  
2   grid-template-columns: 1fr 2fr 1fr;  
3   gap: 16px;  
4 }
```

The gaps are paid first. Then **fr** shares what is left: 1 part, 2 parts, 1 part.

► **ONE UNIT** — **fr** is a *fraction* of the leftover space. It takes any number, so **2fr** is twice the share of **1fr**. A fixed size such as **200px** is paid before any **fr**.

SIZING

DEEPER · SELF-STUDY

fr IS NOT PERCENT WITH A NICER NAME

same intent, two units, one gap slider

↺ reload

open ↗

gap 0px ☐ trim the percentages to 32%

grid-template-columns: 33.333% 33.333% 33.333%

one

two

three

grid-template-columns: 1fr 1fr 1fr

one

two

three

container inner width1498px

gap x 20px

percent resolved to499.328px 499.328px 499.328px

and the row then wants1498px (0)

fr resolved to499.328px 499.328px 499.344px

With no gap these are the same layout, and you should say so plainly.

Nothing so far distinguishes the two units.

Now drag the gap. Every real design has one

▶ **HANDS-ON** `demos/03-fr-vs-percent/` — gap at 0, then drag it in one motion.

CYCLE 1 · BEFORE YOU TYPE

minmax() AND repeat()

A FLOOR AND A CEILING

```
grid-template-columns:  
  minmax(10rem, 1fr) 2fr;
```

`minmax(min, max)`: the track is never narrower than `min` and never wider than `max`. The first column here is at least `10rem`.

THE SAME TRACK, N TIMES

```
repeat(3, 1fr)  
/* = 1fr 1fr 1fr */  
  
repeat(2, 1fr 140px)  
/* = 1fr 140px 1fr 140px */
```

`repeat(n, pattern)` writes a pattern `n` times. The pattern can be several tracks, not only one.

TRACK SIZING

VALUE	MEANS	REACH FOR IT WHEN
200px	exactly that	the thing has a real size — an icon, a control
25%	a share of the container, ignoring gap	almost never in a grid. Use fr
1fr	a share of what is <i>left over</i>	the default answer for a flexible column
auto	content size, then stretches into spare room	a label column beside a field
min-content	as narrow as possible without overflowing	a column that should hug its longest word
max-content	as wide as it wants, wrapping nothing	rarely, and never for prose
minmax(a, b)	a floor and a ceiling	whenever a track has both a limit and freedom
fit-content(a)	content size, capped at a	a caption column that must not run away

THE SIZING VOCABULARY, AS ARITHMETIC

Fixed tracks and gaps are paid first. `fr` shares what is left — and at a narrow width there may be nothing left.

TEMPLATE, GAP: 16PX	AT 1000PX	AT 400PX
200px 1fr 200px	200 · 568 · 200	200 · 0 · 200 — 432 wide, overflows
repeat(3, 1fr 140px)	six tracks: 166.67 and 140, alternating	the three <code>fr</code> tracks are 0 — 500 wide
repeat(3, minmax(10rem, 1fr))	322.67 × 3	160 × 3 — the floor holds, 512 wide

`repeat()` takes a **pattern**, not one track. A floor that does not fit is an overflow, not a squeeze — which is the reason `auto-fit` exists.

TURN THE LINES ON

1 Elements panel: the container has a `grid` badge next to it.

2 Click it. The tracks are drawn on the page, numbered from the start and from the end.

3 Layout pane: *Show line numbers, Show track sizes, Show area names.*

The badge is how you debug a grid: you stop guessing where line 3 is.

```
1 .tracks {  
2   grid-template-columns: repeat(2, 1fr 140px);  
3 }  
4  
5 .last {  
6   grid-column: 1 / -1;  
7 }
```

`repeat()` takes a **pattern**, not just one track. And `-1` is the last line, however many there are.

THE LAYOUT PANE

WHERE

Elements · the `grid` badge

Layout · *Show line numbers*

Layout · *Show line names* · *Hide line labels*

Layout · *Show track sizes*

Layout · *Show area names*

Layout · *Extend grid lines*

WHAT IT SHOWS

Next to any element with `display: grid` or `inline-grid`. Click it to toggle the overlay on the page

The default: every line numbered from the start and, negatively, from the end — where `-1` comes from

Named lines instead of numbers, or no labels at all

What you wrote and what the browser computed, side by side — `1fr` next to `560px`

The picture you drew with `grid-template-areas`, written on the page

The lines run to the edge of the viewport, for checking alignment across regions

▶ **ALSO** — every grid on the page is listed in the Layout pane with its own overlay colour. Flex containers get a `flex` badge the same way.

CYCLE 1 · 30 MIN

FLEX LOSES, GRID WINS: THE SHELL, ITS TRACKS AND ITS FLOOR

Last week's page in honest Flexbox, until a designer asks for a gap. Then the same five regions as a grid: one column that looks like nothing, a track list with commas that is thrown away in silence, percentages that forget the gap, and a URL that swells a column.

1 I DO

16 MIN

9 on the slides above, 7 in the editor · live-code/01-flex-loses-grid-wins · a gap · display: grid · commas · 25% 50% 25% · 1fr 2fr 1fr · 1 / -1 · a long URL · minmax(0, ...) · console lines: **copy this**

2 WE DO

5 MIN

you type it on your copy · compare with checkpoint 1 · the grid badge draws four lines

3 YOU DO

7 MIN

warmup/ — six columns in a pattern of three, and one that goes all the way across

4 DEBRIEF

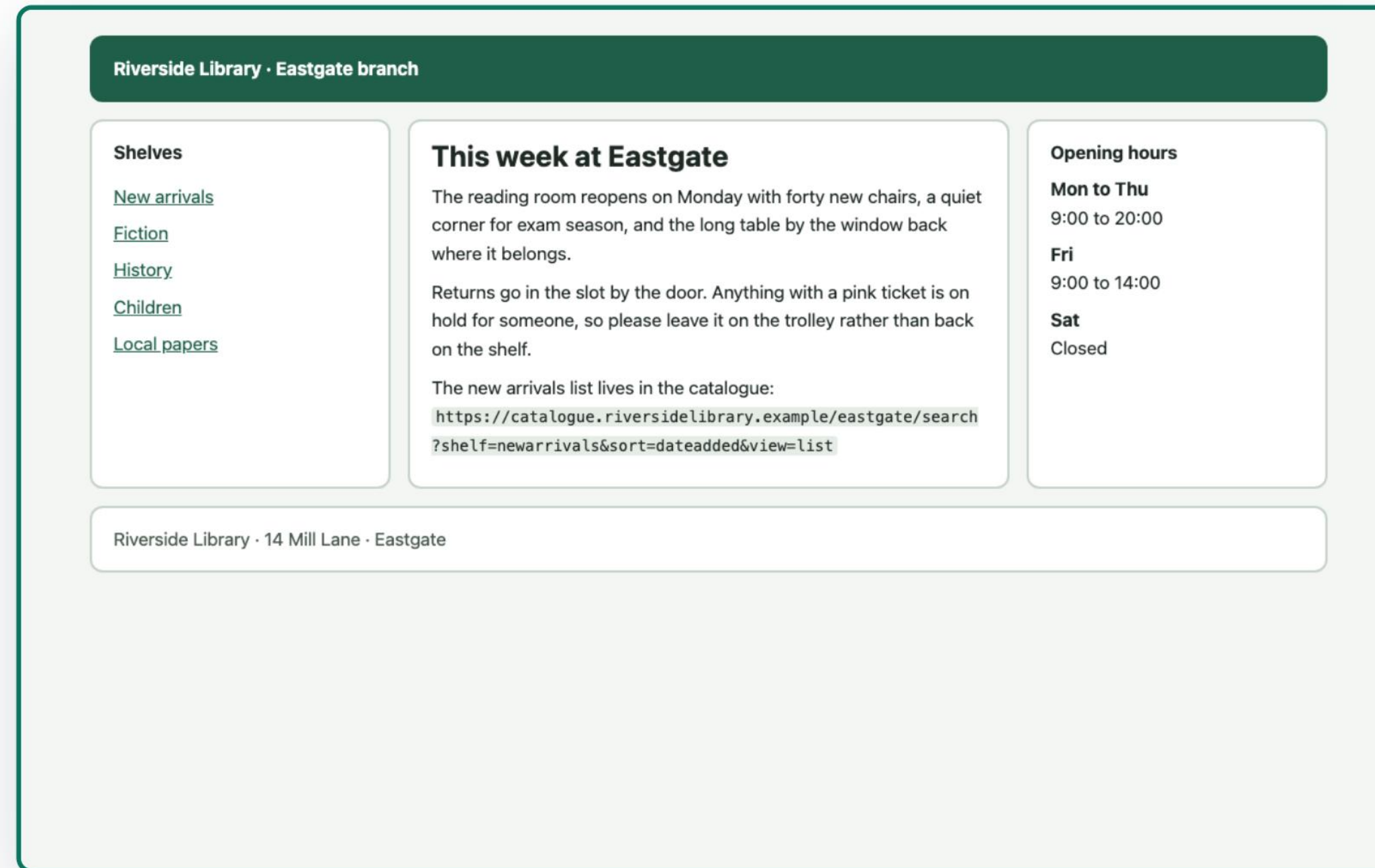
2 MIN

one column is not "Grid does not work" · 1fr is minmax(auto, 1fr) · the Deeper slides

YOU DO · AI-FREE

CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/01-flex-loses-grid-wins/checkpoint.png` is this picture at 1280; `live-code/01-flex-loses-grid-wins/end/` is the code behind it. Five regions, the three in the middle ending on one line. At 768 nothing scrolls sideways, even with the URL. The grid badge draws four column lines.

WHEN A TRACK LIST IS WRONG, NOTHING TURNS RED

A STRAY COMMA

```
1 .row {  
2   grid-template-columns: 1fr, 1fr, 1fr;  
3 }
```

The browser drops the declaration. You get the default: one column.

THE FIX

```
1 .row {  
2   grid-template-columns: 1fr 1fr 1fr;  
3 }
```

Track lists are space-separated. Commas live inside `repeat()`.

► **THE POINT** — no console error, no red, no 404. It just looks like Grid did nothing.

THE TOOL YOU WILL USE ALL EVENING

demos/02-tracks – type a track list, watch the browser answer

reload

open

200px 200px 200px

33% 33% 33%

1fr 1fr 1fr

1fr 2fr 1fr

auto 400px 1fr

repeat(3, 1fr)

repeat(4, minmax(120px, 1fr))

min-content max-content 1fr

grid-template-columns

1fr 2fr 1fr

gap

10px

items

6

THE TRACKS, AND THE LINES THAT COME WITH THEM

1 / -4

2 / -3

3 / -2

4 / -1

1

2

3

4

5

6

3 tracks, so 4 lines, numbered 1 to 4.

you wrote1fr 2fr 1fr

browser used370px 740px 370px

gap10px

items6 in 2 rows

Read the second line, not the first. That is what the layout actually resolved to, in pixels – the USED value. Every track you wrote as a

▶ **HANDS-ON** demos/02-tracks/ — start on 200px 200px 200px and resize the window.

18

← TAUGHT · 17

1fr IS minmax(auto, 1fr)

BURSTS ON A LONG URL

```
1 .layout {  
2   grid-template-columns: 200px 1fr;  
3 }
```

THE FLOOR IS LOWERED

```
1 .layout {  
2   grid-template-columns: 200px minmax(0, 1fr);  
3 }
```

`auto` as a minimum means "as wide as the content". One long word, such as a URL, sets a floor the column will not go under; `0` removes the floor.

► **SAY IT OUT LOUD** — this course's own slide theme puts `min-width: 0` on every `.cols` child for exactly this reason.

YOU HAVE MET THIS BUG. IT IS THE SAME BUG.

why 1fr overflows, and what minmax(0, 1fr) fixes

↺ reload

open ↗

☐ paste a long URL into the content ☐ try overflow: hidden ☐ try a narrower sidebar`grid-template-columns: 200px 1fr`

sidebar

Ordinary text wraps, so nothing is wrong until something cannot wrap.

`grid-template-columns: 200px minmax(0, 1fr)`

sidebar

Ordinary text wraps, so nothing is wrong until something cannot wrap.

```
container inner width      1498px
200px 1fr                  wants 1498px (0)
200px minmax(0, 1fr)      wants 1498px
resolved, top grid         200px 1288px
resolved, bottom grid      200px 1288px
```

Nothing is wrong yet, and nothing will be while the content can wrap.
That is why this bug reaches production: it needs a URL, a long token,
a wide `<pre>` or a table, and none of those are in anybody's test data.

Tick the first box.

► **HANDS-ON** `demos/07-minmax-0/` — try the two wrong fixes before you read the right one.

WHY 1fr HAS A FLOOR, AND THREE WAYS TO LOWER IT

1fr is minmax(auto, 1fr) . For a track, an auto minimum asks each item for its automatic minimum size — its content — unless the item is a scroll container. One unbreakable string, and the column refuses to shrink.

overflow-wrap: break-word does not change that: it breaks the word only after layout. anywhere does, because its break points count toward min-content.

AT 768, CYCLE 1 WITH THE URL	TRACKS
1fr 2fr 1fr	108 · 561 · 112, page scrolls 69px
min-width: 0 on the item	172 · 344 · 172
overflow: hidden on the item	172 · 344 · 172, and it clips
minmax(0, 2fr) on the track	172 · 344 · 172, for every item

CYCLE 2 · BEFORE YOU TYPE

PUT AN ITEM EXACTLY WHERE YOU WANT IT

auto-placement

An item you do not place goes into the first free cell, in order. Until now, that was every item.

grid-column, grid-row

Each takes **start / end** lines:

`grid-column: 1 / 3` covers two columns.

grid-area

All four lines in one value, and it starts with the **row**: *row-start / column-start / row-end / column-end*.

```
1 .sun {  
2   grid-area: 1 / 1 / 3 / 2;  
3 }  
4  
5 .sky {  
6   grid-column: span 2;  
7 }
```

`.sun`: rows 1 to 3, column 1 to 2. `.sky` names no line, only a width of two tracks, and auto-placement finds the room.

PLACEMENT

DEEPER · SELF-STUDY

PUTTING AN ITEM SOMEWHERE ON PURPOSE

demos/05-placement/01-lines

↺ reloadopen ↗

column-start1column-end3row-start1row-end2column-end = -1toggle 4 / 6 columns

A 4 X 3 GRID. ONE ITEM IS PLACED; THE REST FLOW AROUND IT.

1

1

placed

2

1

3

2

4

11

```
.placed {
  grid-column: 1 / 3;
  grid-row: 1 / 2;
}

/* the same placement, written with span */
.placed {
  grid-column: 1 / span 2;
}
```

▶ **HANDS-ON** demos/05-placement/01-lines/ — set 1 / 2 and work out why it covers one column.

FOUR WAYS TO SAY THE SAME PLACEMENT

```
1  .a {
2    grid-column: 1 / 3;
3    grid-row: 2 / 4;
4  }
5
6  .b {
7    grid-column: 1 / span 2;
8    grid-row: 2 / span 2;
9  }
10
11 .c {
12   /* row-start / column-start / row-end / column-end */
13   grid-area: 2 / 1 / 4 / 3;
14 }
15
16 .d {
17   grid-column: 1 / -1;
18   /* all the way across, whatever the track count */
19 }
```

PLACEMENT

WRITTEN	MEANS
<code>grid-column: 1 / 3</code>	from line 1 to line 3 — that is two columns
<code>grid-column: 1 / span 2</code>	the same thing, counted as a width
<code>grid-column: 1 / -1</code>	all the way across, whatever the track count
<code>grid-row: 2 / 4</code>	from row line 2 to row line 4
<code>grid-area: 2 / 1 / 4 / 3</code>	row-start / column-start / row-end / column-end
<code>grid-area: main</code>	the area called main in the container's picture

► **THREE TRACKS MEANS FOUR LINES.** You count the edges, not the things.

CYCLE 2 · BEFORE YOU TYPE

A SQUARE WITH NO HEIGHT, AND A PATTERN WITH A PERIOD

NO HEIGHT ANYWHERE

```
.poster {  
  width: min(100%, 30rem);  
  aspect-ratio: 1;  
}  
  
.tiles > * {  
  aspect-ratio: 1;  
}
```

`.tiles > *` is every direct child of `.tiles`.

`aspect-ratio: 1` means width : height is 1 : 1.

`min(100%, 30rem)` is the smaller of the two, so the poster fills a phone and stops at 30rem. The height comes from the ratio; a fixed height is right at one width only — a layout region never gets one.

COUNT TO WHERE IT REPEATS

```
/* 4 across: odd tiles line up  
   in columns. Stripes. */  
.tiles > :nth-child(8n + 1),  
.tiles > :nth-child(8n + 3),  
.tiles > :nth-child(8n + 6),  
.tiles > :nth-child(8n + 8) { ... }
```

`:nth-child(an + b)` picks every `a`-th child, starting at `b`; `odd` is 1, 3, 5... Here `a` is how many cells until the pattern comes back. On an even-width board that is two rows.

A CHEQUER'S PERIOD, WORKED

`:nth-child(an + b)` matches positions `b`, `a + b`, `2a + b` ... with `n` counting from 0. `odd` is `2n + 1`.

On a board `w` tiles wide, the dark tiles of row one are 1, 3, 5... and the dark tiles of row two are shifted by one. The pattern comes back after **two rows**: the period is `2w`.

If `w` is **odd**, every row already starts on the other parity, and `odd` is a chequer. If `w` is **even**, `odd` is stripes.

```
/* w = 4, so the period is 8 */
.tiles > :nth-child(8n + 1), /* row 1 */
.tiles > :nth-child(8n + 3),
.tiles > :nth-child(8n + 6), /* row 2 */
.tiles > :nth-child(8n + 8) {
  background: var(--dark);
}

/* w = 5: odd is already a chequer */
.five > :nth-child(odd) { ... }
```

CYCLE 2

25 MIN

A PICTURE MADE OF TRACKS

A poster from a sketch in pixels, and a tiled floor. The pixels become fractions, the square stops needing a height, every block is placed by line — and the obvious selector for the dark tiles draws stripes.

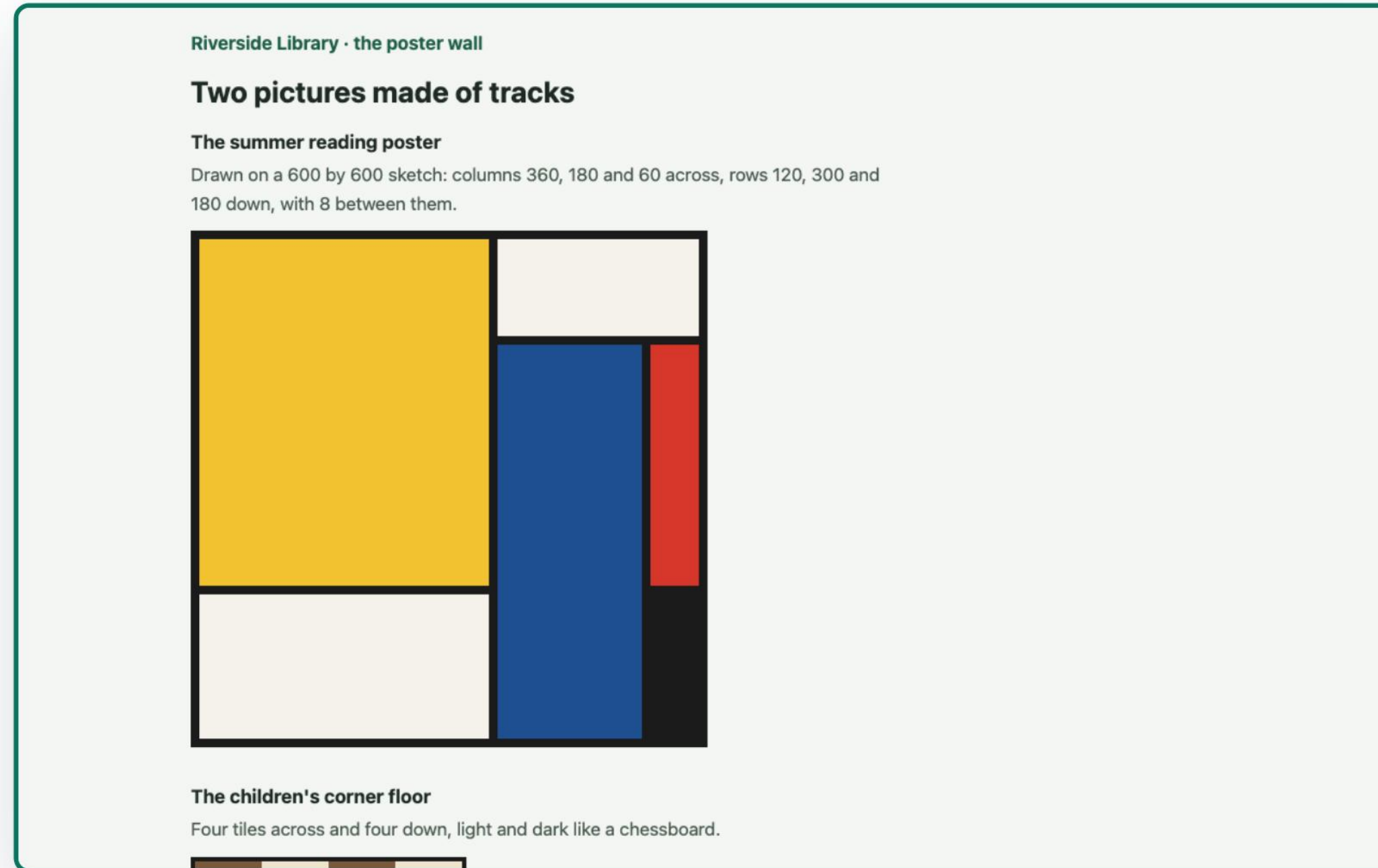
1 I DO 11 MIN 4 on the slides above, 7 in the editor · live-code/02-picture-from-tracks · 360px 180px 60px · fr + height: 30rem · aspect-ratio: 1 · grid-area row / column / row / column · grid-column: span 2 · repeat(4, 1fr) · :nth-child(odd) · :nth-child(8n + 1) · the paste step and the console lines are copy this	2 WE DO 5 MIN you type it on your copy · compare with checkpoint 2 · both still square at 375	3 YOU DO 7 MIN the six-wide rug, and move the red dot · AI as tutor, on why, not on the answer	4 DEBRIEF 2 MIN grid-area starts with the ROW · the period of a chequer · the Deeper slides
--	--	---	--

YOU DO

AI AS TUTOR

CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/02-picture-from-tracks/checkpoint.png` is this picture at 1280; `live-code/02-picture-from-tracks/end/` is the code behind it. At 375 the poster and every tile are still square, and nothing scrolls sideways. There is no `height:` in the file.

CYCLE 3 · THE ONE EVERYBODY GETS WRONG

A CARD ROW THAT NEEDS NO MEDIA QUERY

```
1 .cards {  
2   display: grid;  
3   grid-template-columns: repeat(auto-fit, minmax(15rem, 1fr));  
4   gap: 1rem;  
5 }
```

As many columns as fit, never narrower than 15rem, sharing what is left.

Four cards or nine, phone or monitor, and not one breakpoint.



AUTO-FIT AND AUTO-FILL, SIDE BY SIDE

demos/04-auto-fit-fill – two sliders, one difference

reloadopen

container width 700px items 8

```
repeat(auto-fill, minmax(140px, 1fr))
```

1

2

3

4

5

6

7

8

```
repeat(auto-fit, minmax(140px, 1fr))
```

1

2

3

4

5

6

7

8

Right now these two are identical, and that is the normal case.

container700px

items8

► **HANDS-ON** [demos/04-auto-fit-fill/](#) — 8 items: **IDENTICAL**. Drag the count down: `auto-fill` keeps the empty tracks, `auto-fit` collapses them and the items grow.

31

DEEPER · 32

THE DIFFERENCE, AS A NUMBER

First the browser counts tracks: `floor((W + gap) / (min + gap))`.
Only then does the keyword decide what happens to the empty ones.

3 ITEMS, MINMAX(11REM, 1FR), GAP 16	TRACKS	AUTO-FILL ITEM	AUTO-FIT ITEM
400px	2	192	192 — identical
800px	4	188, one empty slot	256 , the empty track is 0
1152px	6	178.66, three empty slots	373.33 , three tracks at 0

► **WHAT DEVTOOLS SHOWS** — under `auto-fit` the computed `grid-template-columns` still lists the collapsed tracks, as `0px`. Six tracks, three of them zero.

IMAGES, IN THE ORDER THEY BITE

demos/09-images – press load, then the two toggles in order

↺ reload

open ↗

load the images

☐ aspect-ratio: 3 / 2

☐ object-fit: cover

A CARD ROW, AND THREE PICTURES THAT ARE NOT THE SAME SHAPE

3 : 2

2 : 3

1 : 1

row height now 67px

Nothing is loaded yet. Note the row height – then press the button and watch it change. That change is Cumulative Layout Shift, and it is the thing that moves a button out from under somebody's finger.

```
<!-- srcset and sizes: what the browser does BEFORE it downloads anything -->

```

► **HANDS-ON** `demos/09-images/` — read the **LAYOUT SHIFT** number out loud: how far the page jumps when an image arrives late. Then the two properties: `aspect-ratio` sizes the *box*; `object-fit: cover` fills the box with the *picture* and crops it. The default, `fill`, stretches it.

IMAGES

TOOL	JOB
<code>aspect-ratio</code>	decides how big the box is, and reserves it before the file arrives
<code>width</code> / <code>height</code> attributes	the same reservation, for free, before the CSS loads
<code>object-fit: cover</code>	fills the box and crops. For photographs
<code>object-fit: contain</code>	fits inside and leaves space. For logos and diagrams
<code>srcset="... 800w"</code>	the candidate files, each labelled with its real width
<code>sizes="(min-width: 60em) 33vw, 100vw"</code>	how wide the slot will be, so the browser can choose before layout

▶ **TWO PROPERTIES, TWO JOBS.** `aspect-ratio` sizes the box; `object-fit` handles the picture inside it.

aspect-ratio AND THE WIDTH AND HEIGHT ATTRIBUTES

`<img width="600" height="400">` is not a size any more. The browser maps the two attributes to `aspect-ratio: auto 600 / 400`, so with `width: 100%` and `height: auto` the box is reserved before the file arrives, and nothing jumps.

`auto` means "the picture's own ratio once it loads". A bare `aspect-ratio: 3 / 2` overrides it: every box is 3 : 2, whatever the file.

At least one of `width` and `height` must be automatic, or `aspect-ratio` does nothing.

OBJECT-FIT	THE PICTURE INSIDE THE BOX
<code>fill</code> (default)	stretched to the box — distorted
<code>cover</code>	fills the box, cropped
<code>contain</code>	all of it, with empty bands
<code>none · scale-down</code>	natural size · the smaller of none and contain

`object-position` chooses which part survives the crop.

CYCLE 3 · 28 MIN

A CARD ROW THAT NEEDS NO QUERY

Five events that will be seven next week, and three book covers in three shapes. The row decides its own column count, the covers share one shape without being squashed, and the inside of each card stays Flexbox.

1 I DO 11 MIN

4 on the slides above, 7 in the editor · live-code/03-card-row-no-query · repeat(3, 1fr) at 375 · repeat(auto-fit, minmax(15rem, 1fr)) · auto-fill, then auto-fit · aspect-ratio: 2 / 3 · object-fit: cover · flex inside the card

2 WE DO 6 MIN

you type it on your copy · compare with checkpoint 3 · zero @media in the file

3 YOU DO 9 MIN

the staff row: no query, square photos that are not squashed, links on one line

4 DEBRIEF 2 MIN

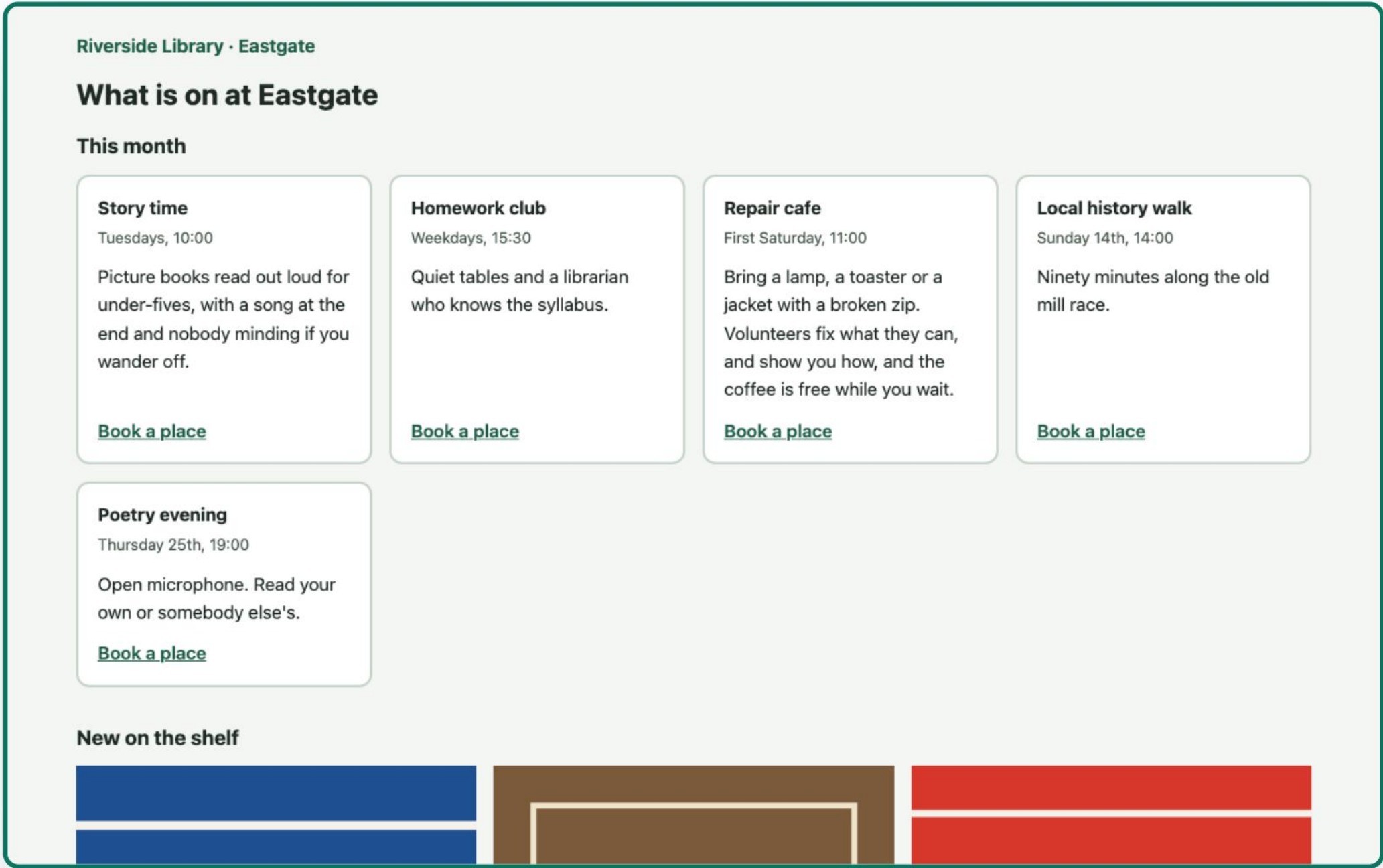
aspect-ratio without object-fit · Grid for the row, Flex for the card · the Deeper slides

YOU DO · AI-FREE



CHECKPOINT 3

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/03-card-row-no-query/checkpoint.png` is this picture at 1280; `live-code/03-card-row-no-query/end/` is the code behind it. At 375 everything is one column and nothing scrolls sideways. Every cover is 2 : 3 and not one is squashed. `@media` appears zero times in the file.

THE RULE, AND IT IS SHORT

GRID

The **page**. Anything with rows and columns that have to line up.

Shells, card grids, dashboards, galleries, forms with a label column.

FLEX

The **inside of a component**. One axis, and the sizes come from the content.

Navbars, button rows, a card's internals, toolbars, a chip with an icon.

NOT A COMPETITION

You don't have to choose one over the other — Flexbox can be used inside a grid cell.

GRID OR FLEX

YOU ARE BUILDING	USE	BECAUSE
a page shell	Grid	rows and columns must line up together
a card row of unknown length	Grid	<code>auto-fit</code> removes the media query
a dashboard, a gallery	Grid	two dimensions, and items that span
a navbar	Flex	one axis, sizes from content
a row of buttons	Flex	one axis. Grid here is the classic overreach
the inside of a card	Flex	a column, with one child taking the slack
a chip with an icon	Flex	two things, centred, on one axis
a form with a label column	Grid	the labels must align across rows

You don't have to choose one over the other — Flexbox can be used inside a grid cell.

ALIGNMENT, IN BOTH DIRECTIONS AT ONCE

THE TRACKS INSIDE THE CONTAINER

`justify-content` ·
`align-content`

Used when the tracks are narrower than the container.

THE ITEMS INSIDE THEIR CELLS

`justify-items` · `align-items`

And `justify-self` /
`align-self` for one item.

THE GOTCHA FROM WEEK 4

In Flexbox, `justify-` follows `flex-direction`. In Grid it is **always** the inline axis. It does not turn.

► **SHORTHAND** — `place-items: center` is both.

ALIGNMENT — THE WHOLE MATRIX

PROPERTY	ALIGNS	ALONG
<code>justify-content</code>	the tracks inside the container	inline (horizontal)
<code>align-content</code>	the tracks inside the container	block (vertical)
<code>justify-items</code>	every item inside its cell	inline
<code>align-items</code>	every item inside its cell	block
<code>justify-self</code>	one item inside its cell	inline
<code>align-self</code>	one item inside its cell	block
<code>place-items</code>	<code>align-items</code> then <code>justify-items</code>	
<code>place-content</code>	<code>align-content</code> then <code>justify-content</code>	

► **UNLIKE FLEXBOX** — these axes never swap. `justify-` is always inline, whatever else you change.

GRID OR FLEX

DEEPER · SELF-STUDY

THE SAME CARD ROW, THREE WAYS

demos/06-three-ways – one width slider, three containers

container width

760px

cards

6

FLEXBOX — flex: 1 1 200px; flex-wrap: wrap

Latency

A slightly longer line of body copy.

view

Errors

A shorter note.

view

Traffic

A slightly longer line of body copy.

view

Saturation

A slightly longer line of body copy.

view

Uptime

A shorter note.

view

Cost

A slightly longer line of body copy.

view

GRID — repeat(3, 1fr)

Latency

A slightly longer line of body copy.

view

Errors

A shorter note.

view

Traffic

A slightly longer line of body copy.

view

Saturation

A slightly longer line of body copy.

view

Uptime

A shorter note.

view

Cost

A slightly longer line of body copy.

view

▶ **HANDS-ON** demos/06-three-ways/ — find a width where all three agree, then break it.

42

← TAUGHT · 38

BREAK

10 MINUTES

Back at 100. Cycle 4 is the AI cycle — have a chat window open, free or paid, Live Server running, and the device toolbar ready.

CYCLE 4 · BEFORE YOU START

WHERE THE NUMBERS COME FROM

So far everything adapted by itself. A media query is for what the content cannot decide: **this is a phone, so put the nav away.**

NOT THIS

```
/* phone */ 480px  
/* tablet */ 768px  
/* laptop */ 1024px
```

A table of devices from 2015. None of those phones exists any more.

THIS

Narrow the window until **your** content stops working. That width is the breakpoint, and it moved when the content changed.

Write it in `em`, so it moves with the reader's font size too.

► **SAY IT OUT LOUD** — you are designing for content, not for a catalogue of hardware.

RESPONSIVE — THE UNITS

UNIT	RELATIVE TO	USE FOR
rem	the root font size — the reader's own setting	type and spacing (media queries use em, the next row)
em	the element's own font size	a media query, so it moves with the reader
ch	the width of a "0"	max-width on prose
vw	1% of the viewport width	inside clamp(), never alone
dvh	the viewport height as it is now	full-height layouts on a phone
vh	the largest viewport height	rarely — the address bar will cover the bottom

CYCLE 4 · 25 MIN

"GIVE ME A RULE OF THUMB"

A "what's on" page in the week-5 style: a card row, a shelf, a sidebar. You ask for a rule of thumb, not for code. The rule sounds right in general. Then you test it on your page.

1 I DO7 MIN 2 on the slide above, 5 in the chat · ai-lab/04-rule-of-thumb · P1 live, write the rule as one sentence that could be false, then measure it at 700, 1100 and 1440 · transcript.md if the room got something else	2 WE DO6 MIN everyone pastes P1 · write the rule as a sentence · make the change it suggests and read the numbers, before the catch is named	3 YOU DO10 MIN write P2 with your own measured numbers · hunt one more edge · the journal line, after the exemplar	4 DEBRIEF2 MIN a rule of thumb is true at one edge, so write it as a sentence that can fail · the journal exemplar
---	--	--	--

YOU DO · AI AS TUTOR



CHECKPOINT 4

THREE QUESTIONS BEFORE THE NEXT CYCLE

1. Did you write the rule as **one sentence with a number or a width in it** — one a measurement could prove false — *before* you opened the browser?
2. Did you measure it on your own page at three widths or more, plus one more condition (fewer items, more items, or a long word), and read numbers instead of looking?
3. Does your journal line hold the rule, **a width and a measurement**, and a verdict that says where the rule is true — and not just "it worked / it didn't"?

Compare — `ai-lab/04-rule-of-thumb/checkpoint-he.md` has the same three questions; `artifact/start/` and `artifact/swap-auto-fit/` are the two pages, ready to open side by side. The shape of the journal line is `ai-exemplars/exemplars/prompt-journal-good.md`, last section.

CYCLE 5 · BEFORE YOU TYPE

DRAW THE LAYOUT, THEN GIVE EACH REGION A NAME

on the container `grid-template-areas`: one quoted string per row, one word per cell. The same word in neighbouring cells makes one named **area**.

a dot

`.` is a cell left empty on purpose.

on each item

`grid-area: <name>` and nothing else: the item asks for its name, the picture decides where it goes.

```
1 .page {  
2   grid-template-areas:  
3     "head head head"  
4     "side main aside"  
5     "foot foot foot";  
6 }  
7  
8 .masthead { grid-area: head; }  
9 .sidebar { grid-area: side; }
```

Three strings, three rows, three words each: a picture of the page.

OR DRAW THE LAYOUT AND LET THE ITEMS CLAIM A NAME

[demos/05-placement/02-areas](#) – the picture is the layout [reload](#) [open](#)

desktop

tablet

phone

a gap in the grid

not a rectangle

ragged rows

grid-template-areas

```
"masthead masthead masthead masthead"
"sidebar main main aside"
"sidebar main main aside"
"footer footer footer footer"
```

THE LAYOUT THAT PICTURE PRODUCES

masthead

sidebar

main

aside

footer

Valid. Every row has the same number of cells and every named area is a rectangle.

A dot is a deliberately empty cell – useful, and the reason "a gap in the grid" is a preset here rather than a mistake.

Note what the five regions themselves say: ``grid-area: main`` and nothing else. The picture holds the layout; the items only claim a name. That is what makes a responsive page shell three pictures in three media queries

► **HANDS-ON** [demos/05-placement/02-areas/](#) — edit a row, then press **PHONE**, then press **NOT A RECTANGLE**.

CYCLE 5 · BEFORE YOU TYPE

TWO RULES AREAS WILL NOT FORGIVE, AND WILL NOT MENTION

RAGGED ROWS

```
"head head head"  
"side main main aside"  
"foot foot foot"
```

Every row needs the same number of cells.

NOT A RECTANGLE

```
"side main main aside"  
"side main aside aside"
```

A named area must be a rectangle.
There is no L shape.

WHAT HAPPENS

The browser drops the declaration and you get plain auto-placement, as in cycle 1. **Nothing is logged.**

CYCLE 5 · BEFORE YOU TYPE

A PAGE AS TALL AS THE WINDOW: THE `1fr` ROW

- 1 `min-height: 100dvh` — the frame is at least as tall as the window. `100dvh` is 100% of the window height *as it is right now*.
- 2 `auto` rows are as tall as their content: the masthead and the footer.
- 3 The middle row is `1fr`: it takes everything that is left.

On an empty page the footer stays at the bottom of the window.

```
1 .frame {  
2   min-height: 100dvh;  
3   grid-template-rows: auto 1fr auto;  
4 }
```

`dvh`, not `vh`: on a phone `100vh` is taller than the window when the address bar shows.

CYCLE 5 · 20 MIN

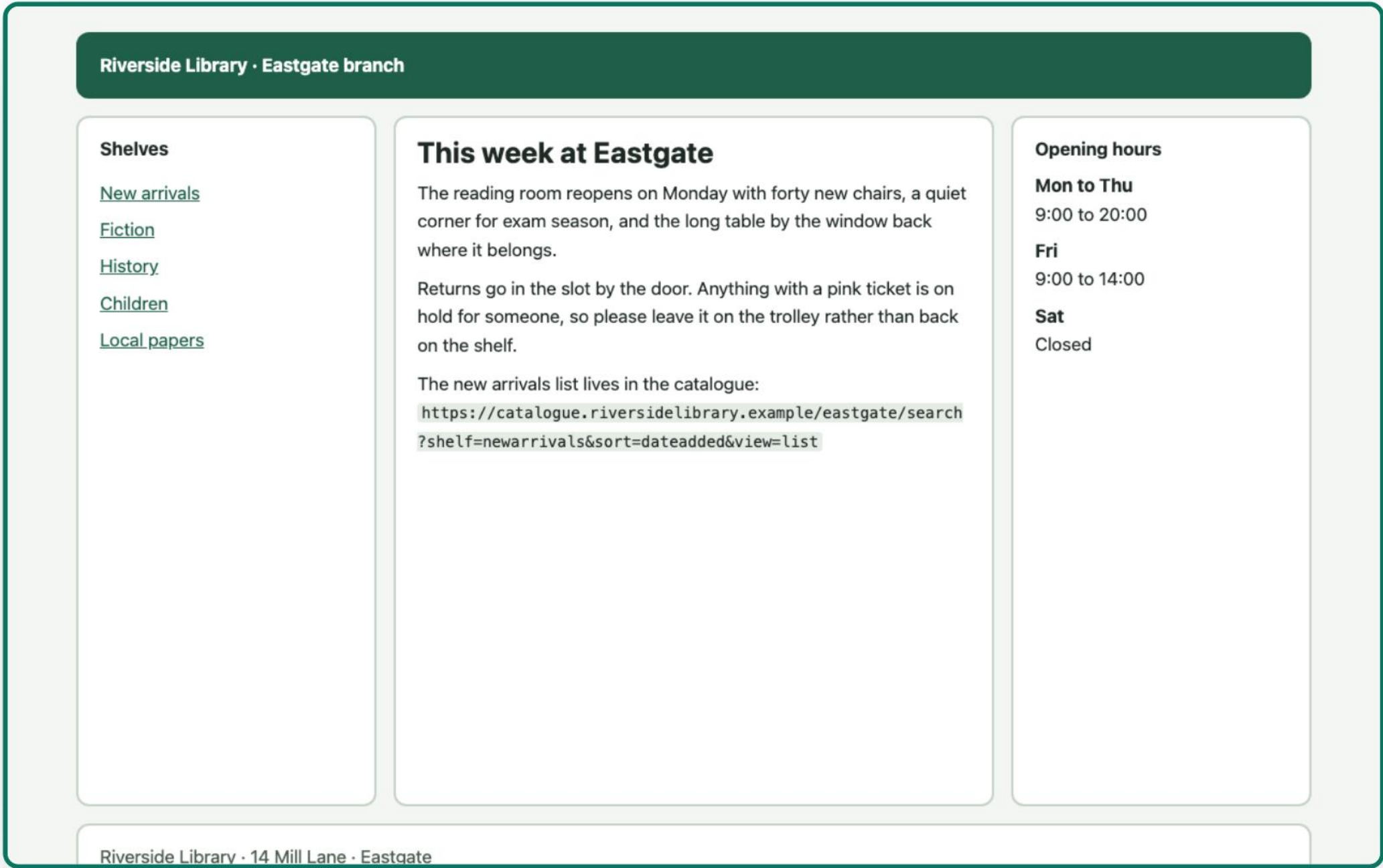
DRAW IT, THEN BUILD IT FROM THE PHONE UP

Cycle 1's page, on a phone: three narrow columns, about 74, 148 and 74 pixels. Rewritten as a picture of names, one column with no query, and two min-width queries that each redraw only the picture — and one row short a cell, thrown away in silence.

1 I DO9 MIN 4 on the slides above, 5 in the editor · live-code/05-draw-then-mobile-first · grid-template-areas, one column · grid-area: name on every region · @media (min-width: 40em) · 64em with a ragged row · the row completed · the page-height lines · paste step: copy this	2 WE DO4 MIN you type it on your copy · compare with checkpoint 5 · 375, 768 and 1280	3 YOU DO5 MIN polish: a fluid h1 with clamp() and a rem term, a hover that moves only transform · AI as tutor	4 DEBRIEF2 MIN count the cells in every row · min-width composes, max-width fights · the Deeper slides
---	---	---	--

CHECKPOINT 5

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/05-draw-then-mobile-first/checkpoint.png` is this picture at 1280; `live-code/05-draw-then-mobile-first/end/` is the code behind it. The columns are stretched to the window height by the `1fr` row; scroll and the footer is below. At 768: two columns and the hours below. At 375: one column, the reading region before the shelves. Every width query is `min-width`.

CYCLE 5 · YOU DO

TYPE THAT DOES NOT STEP: `clamp()`

RIGHT

```
1 h1 {  
2   font-size: clamp(1.6rem, 1rem + 2.2vw, 2.6rem);  
3 }
```

WRONG

```
1 h1 {  
2   font-size: clamp(1.6rem, 4.2vw, 2.6rem);  
3 }
```

The `rem` term keeps the reader's own font setting in the answer. Pure `vw`. Somebody who set a larger default font is ignored entirely.

► **CLAMP(MIN, PREFERRED, MAX)** — the size grows with the window, but never below `min` or above `max`. `vw` is 1% of the window width.

TYPE THAT DOES NOT STEP: THE DEMO

demos/08-responsive/02-clamp

viewport  480px ☒ keep the rem term in clamp() ☐ user has set a larger default font

font-size: 40px

Layout that fits the page

THREE STEPS, THREE MEDIA QUERIES

Layout that fits the page

CLAMP(24PX, 1REM + 2.5VW, 44PX)

Layout that fits the page

viewport 480px

root font-size 16px

clamp(24px, 1rem + 2.5vw, 44px)

minimum 24px

preferred 28.0px ← the PREFERRED value is winning

```
1 h1 {
2   font-size: clamp(1.6rem, 1rem + 2.2vw, 2.6rem);
3 }
```

POLISH, AND THE PROPERTY YOU MUST NOT ANIMATE

`demos/10-motion` – tick "busy" (a page that is busy) and run it again ↺ reload ↗ open

run both ☐ make the main thread busy

ANIMATING inset-inline-start AND width — LAYOUT EVERY FRAME

left

ANIMATING transform — COMPOSITOR ONLY

transform

frames drawn	0
average frame rate	not measured yet
longest frame	— (16.7ms is one frame at 60fps)
frames over budget	0

ANIMATE

`transform` · `opacity` — the browser only moves a finished picture: no layout (working out sizes and places), no paint (colouring pixels)

Colour on a short hover: no layout, but a paint every frame. Fine, not free.

DO NOT

`width` · `height` · `top` · `left` · `margin` — and `transition: all`, which covers them too

MOTION

CHEAP

`transform` · `opacity` — and only these two

The compositor moves an existing layer. No layout, no paint.

EXPENSIVE

`width` · `height` · `top` · `left` · `margin` · `padding`: layout and paint, every frame.

`color` · `background` · `box-shadow`: no layout, but a repaint every frame — a shadow's blur is the slow kind.

TRANSITION: ALL

Covers whatever happens to change — including the expensive ones. Nobody who writes it knows what it covered.

```
@media (prefers-reduced-motion: reduce) {  
  :root { --duration: 0ms; }  
}
```

MIN-WIDTH COMPOSES. MAX-WIDTH FIGHTS.

demos/08-responsive/01-breakpoints — same output, two sources

viewport 380px

nav

A headline that has to fit
The width at which this stops fitting is the
breakpoint. Not 768.

aside

MOBILE-FIRST — MIN-WIDTH

```
.page {  
  grid-template-columns: 1fr;  
}  
@media (min-width: 560px) {  
  .page { grid-template-columns: 1fr 1fr; }  
  .nav { grid-column: 1 / -1; }  
}  
@media (min-width: 860px) {  
  .page { grid-template-columns: 200px 1fr 200px; }  
}
```

4 declarations, 1 block(s) applying now

DESKTOP-FIRST — MAX-WIDTH

```
.page {  
  grid-template-columns: 200px 1fr 200px;  
}  
@media (max-width: 859px) {  
  .page { grid-template-columns: 1fr 1fr; }  
  .nav { grid-column: 1 / -1; }  
}  
@media (max-width: 559px) {  
  .page { grid-template-columns: 1fr; }  
  .nav { grid-column: auto; }  
}
```

► Each `min-width` query **ADDS** to the rules above it; each `max-width` query has to **UNDO** them. **HANDS-ON** [demos/08-responsive/01-breakpoints/](#) — drag slowly and watch which blocks light up.

THE QUERY A COMPONENT ASKS ABOUT ITSELF

A media query asks about the **window**. A card cares how wide *it* is — the same card might sit in a sidebar and in a full-width row.

Container queries ask the second question. **Not examined, not needed for the assignment** — but they are why "breakpoints come from content" is more than a slogan.

`container-type: inline-size` makes an element a query container; `@container` asks about the nearest one; `cqi` is 1% of its inline size.

```
1 .panel {  
2   container-type: inline-size;  
3 }  
4  
5 @container (min-width: 30em) {  
6   .card {  
7     grid-template-columns: 8rem 1fr;  
8   }  
9 }
```

ASSIGNMENT · 145–175

THE ASSIGNMENT: ASSEMBLE WHAT YOU TYPED, THEN EXTEND IT

ASSEMBLE · CORE

`week-05/` is in your repo from `pre-class/`. The painting and the board are cycle 2, bigger. The shell is cycles 1 and 5: a picture of names, one column, two `min-width` queries. The card row is cycle 3 — and inside each `.kpi` it is still Flexbox.

EXTEND · STRETCH AND CHALLENGE

The status board is cycle 3's covers at `3 / 2`. The fluid number and the no-layout hover are cycle 5's "you do". Then the challenge: a wide card, the hole it leaves, and `subgrid` behind `@supports` — the next slide explains `@supports` now; the challenge is finished at home.

THE RULES, AND THE EXAM

No `max-width` query. No fixed height on a layout region. Grid for the page, Flex for the component. AI as tutor — it explains, you type. SHA in Moodle. Home assignment 2 is out today. The exam (week 13) asks about this code, offline.

CHALLENGE · FINISHED AT HOME

@supports : NEW CSS, WITH THE OLD VERSION AS THE FLOOR

@supports A **feature query**, like `@media` but about the browser, not the window. The rules inside apply only if the browser understands the declaration in brackets.

subgrid A newer value: a card that is itself a grid takes its rows from the grid it sits in, so every footer lines up.

Write the version that works everywhere **first**. Put the new one inside `@supports`, where it can only add.

```
1 .kpi {  
2   display: flex;  
3   flex-direction: column;  
4 }  
5  
6 @supports (grid-template-rows: subgrid) {  
7   .kpi {  
8     display: grid;  
9     grid-template-rows: subgrid;  
10    grid-row: span 4;  
11  }  
12 }
```

THE GRID YOU DID NOT DECLARE

- implicit grid** Items past the end of your explicit tracks generate new rows. Sized by content unless you say otherwise.
- grid-auto-rows** The size of those rows. Important when items arrive from data.
- grid-auto-flow** **row** by default. **column** flips it.
- dense** Lets a later, smaller item climb back into a hole. Changes visual order, not the DOM — so tab order stops matching the screen.

```
1 .gallery {  
2   display: grid;  
3   grid-template-columns: repeat(3, 1fr);  
4   grid-auto-rows: 180px;  
5 }  
6  
7 .gallery {  
8   grid-auto-flow: dense;  
9 }
```

This is how a list of unknown length lays itself out.

SUBGRID, IN ONE SLIDE

A grid item that is itself a grid normally invents its own rows. With `subgrid` it takes them from the track it sits in — so the third line of every card in a row is the *same* line.

It is the honest answer to "make the footers line up when the cards hold different amounts of text".

```
1  .card {  
2    display: grid;  
3    grid-template-rows: subgrid;  
4    grid-row: span 4;  
5  }  
6  
7  @supports (grid-template-rows: subgrid) {  
8    /* wrap it, and keep the flex version  
9      as the fallback */  
10 }
```

DEBRIEF · 175–180

THREE QUESTIONS BEFORE YOU LEAVE

- 1 Three columns. How many lines, and what is the last one also called?
- 2 Six cards, room for four columns. What is different between `auto-fit` and `auto-fill` here? And with three cards?
- 3 Your sidebar refuses to shrink and the page scrolls sideways. One declaration.

► **GO DEEPER** — the slides tagged *Deeper* are optional self-study, and they are not examined: every track-sizing keyword, placement in four spellings, why `1fr` has a floor, the period of a chequer, alignment, container queries, `aspect-ratio` against the width and height attributes, the Layout pane, and the demos behind today's cycles. The next slide lists them.